

Reviewer Pack — Minimal Generality of Quasi-Polynomial Functors Bounds Circu...

Entry 202dec57ff0f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 03:22:16 UTC

Minimal Generality of Quasi-Polynomial Functors Bounds Circuit Depth for Boolean Functions

Entry ID: 202dec57ff0f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 03:22:16 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Quasi-Polynomial Functors) **Field B** (complexity object): Boolean Function Complexity (Circuit Depth)

Statement:

For every boolean function $f: \{0, \dots, N-1\} \rightarrow R$ with $N \leq 40$, the circuit depth of f is upper-bounded by a constant times the generality of the quasi-polynomial functor Φ_f associated with f . Specifically, $D(f) = \Theta(\text{generality}(\Phi_f))$ where $D(f)$ denotes the circuit depth and $\text{generality}(\Phi_f)$ is defined as the minimum number of parameters required to define Φ_f such that Φ_f maps polynomials over R^N to boolean functions.

Rationale (proposer's reasoning):

Quasi-polynomial functors in category theory are known to capture certain complexity properties, but their connection to circuit complexity has not been thoroughly explored. This conjecture suggests a potential link between the generality of these functors and the circuit depth of boolean functions, which could expose new insights into the structure of boolean functions.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e488d903607970da

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For boolean function $f: \{0, \dots, N-1\} \rightarrow R$ with $N \leq 40$, if circuit depth $D(f)$ is less than or equal to a constant times $\text{generality}(\Phi_f)$, where the constant is determined from prior research and $\text{generality}(\Phi_f)$ is the minimum number of parameters for Φ_f such that it maps polynomials over R^N to boolean functions.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quasi-Polynomial Functors" AND "Boolean Function Complexity" AND circuit depth - "generality(Φ_f)" AND circuit depth bound AND Category Theory - "circuit depth" ~ "generality of Quasi-Polynomial Functors"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(N):
        return [random.choice([0, 1]) for _ in range(2**N)]

    def compute_generality(phi_f):
        # Placeholder function to compute generality
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, N)

    def construct_circuit(f, N):
        # Placeholder function to construct circuit
        # This is a dummy implementation and should be replaced with actual logic
        return len(f) # Simplified for demonstration

    results = []
    n_values = [5, 10, 15, 20, 30, 40]

    for N in n_values:
        f = generate_boolean_function(N)
        phi_f = compute_generality(f)
        depth = construct_circuit(f, N)

        results.append({
            "N": N,
            "f": f,
```

```

        "phi_f": phi_f,
        "depth": depth
    })

    if not results:
        return {
            "metric_name": "circuit_depth",
            "metric_value": 0.0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_instances_generated"
        }

    total_depth = sum(result["depth"] for result in results)
    avg_depth = total_depth / len(results)
    n_max = max(result["N"] for result in results)

    return {
        "metric_name": "circuit_depth",
        "metric_value": avg_depth,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if not trial_result["conjecture_holds"]:
            results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16124
2	preregistration	ollama_remote	glm4:latest	0	0	10293
3	novelty	ollama_remote	glm4:latest	0	0	17662
4	novelty	ollama_remote	glm4:latest	0	0	12906
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14580
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22918
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24415
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12912
9	judge	ollama_remote	glm4:latest	0	0	24368

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 156178 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/202dec57ff0f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/202dec57ff0f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/202dec57ff0f.tar.gz (if generated)